FINAL YEAR PROJECT REFERENCE DOCUMENT

CampusHub

Academic & Campus Collaboration System — Overview

TEAM MEMBER

V. Karthikeyan (24UCS025)

BSc Computer Science 'A'

PROJECT GUIDE

K. Devikala M.C.A.,

Department of Computer Science

1. Project Description

CampusHub is an integrated college collaboration platform built for students, faculty, and administrators. The platform centralizes academic resources, campus communication, placement opportunities, and event management into a single, role-based web application, replacing the fragmented mix of chat groups, physical notice boards, and disconnected spreadsheets that most colleges rely on today. Built on the MERN stack (MongoDB, Express.js, React, and Node.js) with Socket.io for real-time features, CampusHub issues each user a JSON Web Token (JWT) on login and uses role-based middleware to present a tailored dashboard to Students, Faculty, and Admins — ensuring every user sees only the features relevant to their role. The platform's goal is to reduce everyday friction: finding the right notes before an exam, discovering placement drives a student is actually eligible for, registering for an event without back-and-forth messages, and reaching a classmate directly instead of searching through unrelated group chats.

2. Authentication & Access Control Module

Every user registers with their college register number, name, department, and role. Passwords are hashed with bcrypt before storage, and a successful login returns a signed JWT valid for seven days, carried on every subsequent request to authenticate the user and authorize their role. Role-based middleware restricts each API route so that Students, Faculty, and Admins only reach the actions appropriate to their role — for example, only Faculty and Admin accounts can upload notes or publish events. A user who forgets their password can recover access through a one-time password (OTP) sent to their registered email via Nodemailer, without ever exposing the original password.

3. Admin & Platform Management Module

Administrators have full oversight of the platform through a dedicated dashboard that aggregates live statistics — total students, faculty, notes, placement drives, and events — pulled directly from the database. From this dashboard, Admins can manage user accounts, moderate notes uploaded by faculty, publish or edit placement drives and campus events, and monitor overall platform activity. This centralized control keeps the quality and appropriateness of shared academic and placement content consistent across the college.

4. Notes Hub Module

The Notes Hub allows verified faculty and administrators to publish syllabus-aligned lecture materials, previous year question papers, lab manuals, and reference notes as PDF, DOCX, or image files. Uploaded files are stored securely on Cloudinary, keeping the primary database lightweight while ensuring fast, reliable downloads. Students can search and filter the Notes Hub by department, academic year, semester, and subject to instantly locate material relevant to them, without needing to know which faculty member uploaded it or when. Every note uploaded here is automatically indexed by the AI Campus Assistant, so its content becomes immediately searchable through natural-language questions.

5. Chat Portal & Connect-Request Module

CampusHub's Chat Portal lets students communicate directly and instantly using Socket.io-powered real-time messaging, but access is controlled through a Connect-Request flow to keep conversations intentional and consent-based. A student searches for a classmate using their exact register number and may send exactly one message as a connection request; the recipient must accept before the conversation unlocks into a normal two-way chat. Either party can block the other at any time, which silently and immediately ends the connection — the blocked party's messages simply stop being delivered, with no error shown, while re-establishing contact afterward requires a fresh request through the search flow. Faculty and Admin accounts are exempt from this request-based gating and can message any user directly, supporting their role in guiding and coordinating with students.

6. Placement & Training Cell (PAT)

CampusHub features a placement management portal where the college placement officer or Admin broadcasts placement drives, job openings, and internship opportunities to the student body. Each listing specifies the company, role, eligible departments, minimum CGPA, and backlog criteria, along with the application deadline and a direct link to the external application form. Students can filter listings by department, status (open or closed), or keyword search across company name and job role, so they only see drives genuinely relevant to their eligibility — reducing the noise of scrolling through opportunities they don't qualify for.

7. Events and Extracurricular Hub

The Events module allows departments, college clubs, and student societies to organize technical symposiums, hackathons, cultural festivals, workshops, and other extracurricular activities, and to publish them for the whole campus to discover. Students can register for free events instantly with a single click, while paid events are secured through Razorpay's payment gateway — the server verifies each payment's cryptographic signature (HMAC-SHA256) before confirming a ticket, preventing tampered or fraudulent payment claims. Every student's Registrations dashboard tracks their ticket status and payment status for every event they have signed up for, in one place.

8. AI Campus Assistant (RAG Chatbot)

CampusHub includes an AI Assistant powered by Google Gemini and Retrieval-Augmented Generation (RAG). The assistant answers student questions about academic notes, placement eligibility, and this project document itself — but strictly from CampusHub's own data, never from open internet knowledge, and never from private chat messages or another user's personal data. Notes, placement listings, and this reference document are split into small text chunks and converted into vector embeddings using Gemini's embedding model; these embeddings are stored directly inside MongoDB rather than a separate vector database, keeping the system simple and self-contained. When a student asks a question, it is embedded the same way and compared against every stored chunk using cosine similarity; the most relevant chunks are retrieved and passed to Gemini, which generates a grounded, concise answer citing the subject

or document it came from. If the answer isn't present in CampusHub's own data, the assistant says so rather than guessing.

9. Message Security (Planned Enhancement)

As part of CampusHub's ongoing security roadmap, chat messages are planned to be encrypted at rest using AES-256-GCM before being stored in MongoDB. The encryption secret key will live only on the server, never exposed to the client, so that even a leaked or compromised database would reveal no readable message content without server-side access. This is server-side encryption-at-rest rather than full end-to-end encryption, chosen deliberately so it remains compatible with planned Admin moderation oversight of the platform.

10. Campus Policies and Guidelines

- **Academic Integrity:** Notes shared must be original or appropriately credited lecture materials; plagiarized or unauthorized copyrighted content is not permitted on the platform.
- **Placement Eligibility:** Students must maintain minimum CGPA requirements with no active standing backlogs unless explicitly permitted by a specific drive's criteria, as set by the placement officer for each listing.
- **Code of Conduct:** Campus communication and discussion must remain respectful, professional, and compliant with campus regulations; harassment, spam, or misuse of the Connect-Request and Chat Portal features may result in account restrictions by an Admin.
- **Data Privacy:** Personal information, private conversations, and connection details are never shared with or accessible to the AI Assistant, third parties, or other students beyond what a user explicitly makes visible, such as by accepting a connection request.